INF221 — Lecture 3

Today's topics

1. Divide-and-conquer: Tromino
2. Analysing the runtime of merge sort
3. Growth of functions
4. Solving recurrence equations
5. Some useful mathematical material

Divide-and-conquer approach

- The **divide-and-conquer** algorithm design can perform the same sorting task (for worst case scenario) in far less time than **insertion sort**
- Many useful algorithms are *recursive*
- Recursion in computer science is a method where the solution to a problem depends on solutions to smaller instances of the same problem
- Recursion is one of the central ideas of computer science

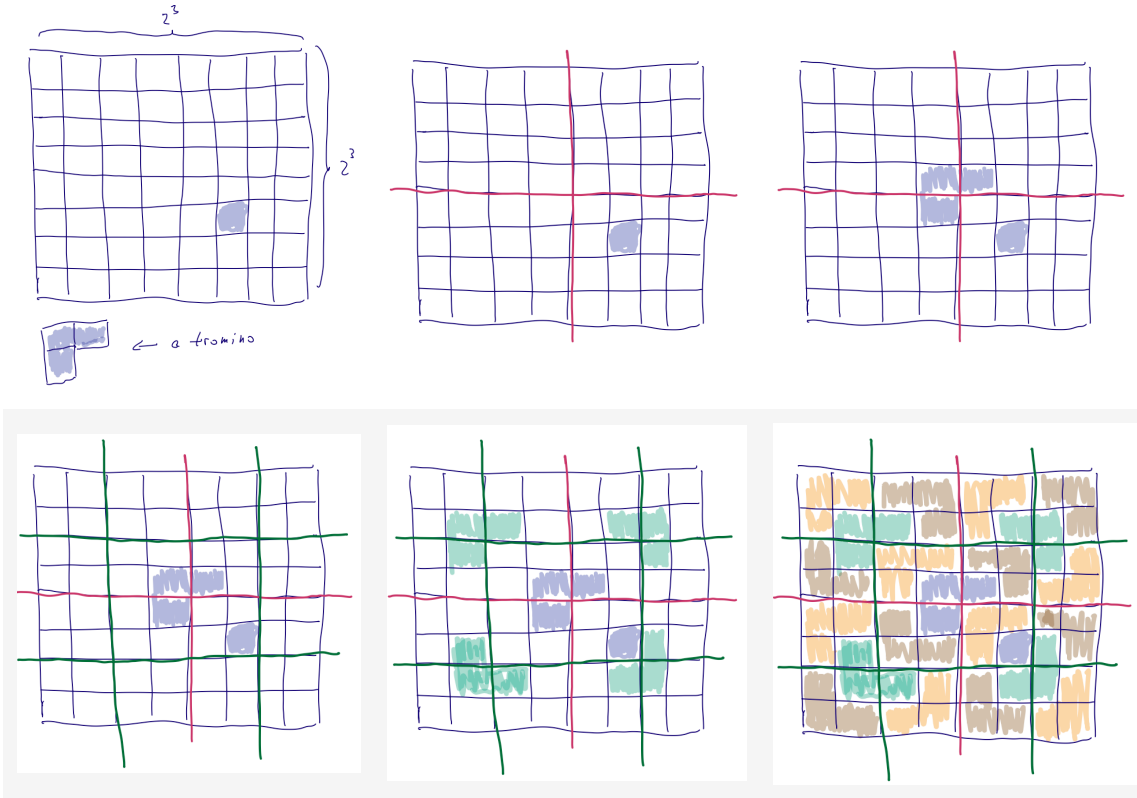
Divide-and-conquer paradigm involves *three* steps at each level of *recursion*.

1. **Divide** the problem into a number of subproblems
2. **Conquer** the subproblems by solving them *recursively*. If the subproblem sizes are small enough, however, just solve the subproblems in a straightforward manner
3. **Combine** the solutions to the subproblem into the solution for the original problem.

Tromino — an example of a problem best solved recursively

- We are given a field with $2^n \times 2^n$ squares, for $n \in \mathbb{N}_0$.
- Exactly one of these squares is filled.
- Fill the remaining squares by placing L-shaped trominos into the field!

See also Wikipedia article on Trominos.



Tromino algorithm

1. Divide the problem evenly into four subproblems. Only one subproblem will contain a filled square.
2. Place a single tromino so that it fills exactly one square in each of the three empty subproblems ("place it around the corner of the subproblem with the filled square").
3. Now all four subproblems contain exactly one filled square, i.e., we have four problems of exactly the same type as the original problem, but with size $2^{n-1} \times 2^{n-1}$.
4. Repeat 1–3 for the subproblems, stopping at 2×2 subproblems.

Comments

- One can visually "see" that the algorithm will work. We will not attempt a formal proof here.
- It is a typical and visual example of breaking larger problems into smaller problems *of exactly the same type* and solving those.
- Because the subproblems are of exactly the same type, we can solve them with exactly the same algorithm.
- This is the power of *recursion* (from Latin "re-" for "again" and "currere" for "to run").
- Recursion is one of the most powerful and important concepts in computer science. We will see many examples during the course. See also the Wikipedia article on

recursion in computer science).

Challenge

A little reflection over the Tromino problem above suggests the following mathematical theorem:

> For every $n \in \mathbb{N}_0$, there exists an $m \in \mathbb{N}_0$ such that $4^n = 3m + 1$.

Can you argue informally why this should hold, and can you give a mathematical proof by induction?

This theorem describes a relation between integers. This leads to quite different mathematics than the calculus of real numbers. Such equations are a part of *discrete mathematics* and are quite important for computer science and cryptography.

Analysing the runtime of merge sort

Merge-sort procedure

- Use *merge* procedure as a subroutine inside merge-sort algorithm
- `merge_sort(A, p, r)` sorts elements in the subarray `A[p..r]`
- If $p \geqslant r$, the subarray has **at most** *one* element and is therefore already sorted
- Otherwise: partition `A[p..r]` into two subarrays `A[p..q]` and `A[q+1..r]` with $q = \lfloor (p+r)/2 \rfloor$

Pseudocode for merge-sort procedure

```
MERGE-SORT(A, p, r)
if p < r
    q = ⌊(p + r)/2⌋
    MERGE-SORT(A, p, q)
    MERGE-SORT(A, q + 1, r)
    MERGE(A, p, q, r)
```

To sort a complete array, call `MERGE-SORT(A, 1, A.length)`.

Floor and Ceiling

- *Floor*: $\lfloor x \rfloor$ is the largest integer $\leq x$
- *Ceiling*: $\lceil x \rceil$ is the smallest integer $\geq x$
- For $x \in \mathbb{Z}$ we have $\lfloor x \rfloor = \lceil x \rceil$

In-class exercise

1. What are $\lfloor \frac{3}{2} \rfloor$, $\lceil \frac{3}{2} \rceil$, $\lfloor -\frac{3}{2} \rfloor$, $\lceil -\frac{3}{2} \rceil$?
2. Show that $\lfloor n/2 \rfloor + \lceil n/2 \rceil = n$ for all $n \in \mathbb{N}_0$!
3. Show that choosing $q = \lfloor (p+r)/2 \rfloor$ yields subarrays $A[p \dots q+1]$ and $A[q \dots r]$ with lengths $\lceil n/2 \rceil$ and $\lfloor n/2 \rfloor$ respectively

---

Analysing divide-and-conquer algorithms

- When algorithm contains recursive call to itself:
  - running time can be often described by ***recurrence equation*** or ***recurrence***
  - ***recurrence*** describes overall running time on a problem of size n
  - does so in terms of the running time on smaller inputs
- Can use mathematical tools to:
  - solve the recurrence
  - provide bounds on the performance of the algorithm
- See Compendium Part 3 and 5

Recurrence for running time of divide-and-conquer algorithm

- $T(n)$ is the running time on a problem of size n
- If problem size small enough:
  - that is, $n \leq c$ for some constant c:
  - straightforward solution takes constant time $\Theta(1)$
- Otherwise: division of problem yields a subproblems
  - each subproblem is of size $1/b$ of the original problem
  - for *merge-sort* both a and b are 2
  - note: for many *divide-and-conquer* algorithms $a \neq b$
  - time needed to solve one subproblem is $T(n/b)$
  - time needed to divide original problem into subproblems is $D(n)$
  - time needed to combine solutions to the subproblems into the solution to the original problem is $C(n)$

The above can be written in the following way

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leq c, \\ aT(n/b) + D(n) + C(n) & \text{otherwise} \end{cases}$$

We will discuss methods for solving recurrences later.

A few notes on recurrence-based analysis

- Note that the pseudocode for *merge sort* works correctly even if number of elements is *not even*
- Recurrence-based analysis is simplified:
  - it assumes that original problem size is a power of 2 (i.e., that number of elements is even) despite the fact that the problem has an odd number of elements
  - this assumption implies that the size of two subsequences is **exactly** $n/2$
  - one can show that this assumption does not affect order of growth of solution of reccurence

Setting up recurrence for $T(n)$ for worst-case running time of merge sort on n numbers

- If merge sort is applied on just **one element** it takes constant time
- If $n > 1$:
  - **Divide**
    - computes the middle of the subarray
    - this computation takes constant time (not dependent on size of subarray)
    - therefore $D(n) = \Theta(1)$
  - **Conquer**
    - recursively solves two subproblems
    - each subproblem is of size $n/2$
    - in sum all subproblems contribute $2T(n/2)$ to the running time
  - **Combine**
    - Already established above that **merge** procedure on an n-element subarray takes time $\Theta(n)$
    - therefore $C(n) = \Theta(n)$

For **merge sort** with $a = 2$, $b = 2$, $D(n) = \Theta(1)$, and $C(n) = \Theta(n)$

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leq c, \\ aT(n/b) + D(n) + C(n) & \text{otherwise} \end{cases}$$

becomes

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1, \\ 2T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases}$$

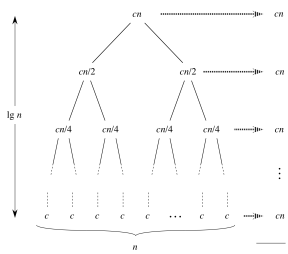
Note that since the sum of $D(n) + C(n) = \Theta(1) + \Theta(n)$ is a linear function of n the result is $\Theta(n)$.

If we use a constant c to represent the time to solve problems of size 1, as well as the time per array element of the divide and combine steps, we can rewrite the recurrence the following way:

$$T(n) = \begin{cases} c & \text{if } n = 1, \\ 2T(n/2) + cn & \text{if } n > 1 \end{cases}$$

Recursion tree

Before we look at formal methods to solve the recurrence above, we can obtain a solution by looking at the recursion tree.



Source: *CLSR Fig 2.5*

- Assumes that n is exact power of 2
- At the root (top level): the cost is cn
- At a specific level i below top:
  - level has 2^i nodes, each contributing a cost of $c(n/2^i)$
  - the total cost across all nodes at this level is $2^i c(n/2^i) = cn$
  - Example: at level $i = 2$ below top
    - there are $2^2 = 4$ nodes
    - the cost for one node is $c(n/4)$
    - the total cost across all nodes is $4 \times c(n/4) = cn$
- Total number of levels is $\lg n + 1$, where n is the number of leaves (corresponding to input size)
- To compute the total cost represented by recurrence the costs at all levels are added up
  - recurrence tree has $\lg n + 1$ levels
  - the cost at each level is cn
  - the total cost: $cn(\lg n + 1) = cn \lg n + cn$
  - Ignoring low-order term and constant gives $\Theta(n \lg n)$

Growth of Functions

Order of growth of running time of an algorithm

- Compendium, Part 4
- Gives a simple characterisation of the algorithm's efficiency

- Allows for comparison of relative performance of alternative algorithms
- Example:
  - **insertion sort** worst-case running time is $\Theta(n^2)$
  - **merge sort** worst-case running time is $\Theta(n \lg n)$
  - for large n, merge sort outperforms insertion sort
- Sometimes exact running time can be determined (as above for **insertion sort**)

$$
\begin{aligned}
T(n) &= c_2 n + c_3(n-1) + c_4(n-1) + c_5 \sum_{j=2}^{n} t_j + c_6 \sum_{j=2}^{n} (t_j - 1) + c_7 \sum_{j=2}^{n} (t_j - 1) \\
&= c_2 n + (c_3 + c_4 + c_8)(n-1) + (c_5 + c_6 + c_7) \sum_{j=2}^{n} t_j - (c_6 + c_7)(n-1) \\
&= c_2 n + (c_3 + c_4 + c_8 - c_6 - c_7)(n-1) + (c_5 + c_6 + c_7) \sum_{j=2}^{n} t_j \\
&= (c_5 + c_6 + c_7) \sum_{j=2}^{n} t_j + (c_2 + c_3 + c_4 + c_8 - c_6 - c_7)n - (c_3 + c_4 + c_8 - c_6 \\
&= A \sum_{j=2}^{n} t_j + Bn + C
\end{aligned}
$$

- **Insertion sort** running time $T(n)$ for worst case (sorted data, $t_j = j$)

$$
T(n) = \frac{A}{2} n^2 + \left(\frac{A}{2} + B \right) n + (C - A) \tag{6}
$$

- However, usually it is not worth the effort to gain extra precision of running time
- Illustrative example: how important is precision of a scale when weighing elephants and mice?
- For large enough inputs, **multiplicative constants** and **lower-order terms** of exact running time are dominated by **effects of input size** itself

Studying asymptotic efficiency of algorithms

- Studying input sizes **large enough** to make only the order of growth of running time relevant
- Concerned with how running time of an algorithm **increases with size of input** *in the limit*, that is size of input increases without bound
- Usually, an algorithm that is **asymptotically** more efficient is the best choice for all but very small input

Asymptotic notation

Asymptotic notation, functions and running times

- We will usually use **asymptotic notation** to characterise the running time of algorithms
- Asymptotic notation can also apply to functions that characterise other aspects of algorithms (such as amount of space they use, etc.)
- However, when applying asymptotic notation to running time of an algorithm:
  - we need to understand **which** running time is meant
  - sometimes we are interested in **worst-case** running time
  - but often we wish to characterise the running time, regardless of input (whether it is worst case or best case, etc.). That is, we wish to make a blanket statement that covers all inputs
  - asymptotic notations are well suited to characterising running times, no matter what the input

Θ-notation

- From above: worst-case running time of insertion sort is $T(n) = \Theta(n^2)$
- What precisely does this mean?

Definition

For a given function $g(n)$, we denote by $\Theta(g(n))$ the following *set of functions*

$$\Theta(g(n)) = \{f(n) | \, \exists \, c_1 > 0, c_2 > 0, n_0 \in \mathbb{N} \ni 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \; \forall \, n \geq n_0\}$$

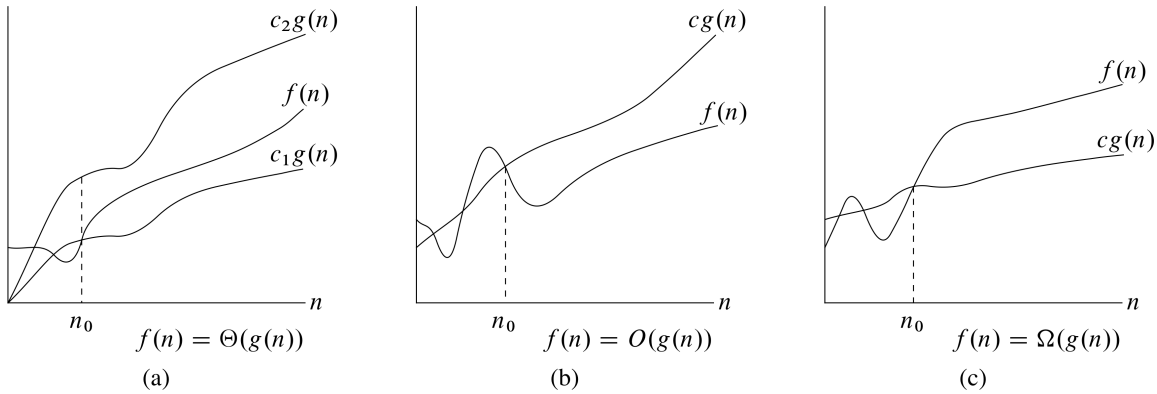
$\exists$	$\ni$	$\forall$
there exists	such that	for all

Or in words: $\Theta(g(n))$ is the set of all functions for which we can find constants $c_1 > 0, c_2 > 0, n_0 \in \mathbb{N}$ so that

$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \qquad \text{for all } n \geq n_0 \; .$$

Even more graphic: A function $f(n)$ belongs to the set $\Theta(g(n))$ if there exist positive constants c_1 and c_2 such that it can be "sandwiched" between $c_1 g(n)$ and $c_2 g(n)$ for all sufficiently large n.

Important: The lower and upper bound are given by the same function $g(n)$.



Source: *CLRS Fig 3.1*

- Because $\Theta(g(n))$ is a set (of functions), we could (should) write $f(n) \in \Theta(g(n))$
- Instead:
  - we will usually write $f(n) = \Theta(g(n))$ to express the same notion
  - doing so simplifies notation and is common practice in the field
- Consider **Figure 3.1a** from CLRS above. It gives an intuitive picture of function $f(n)$ and $g(n)$, where $f(n) = \Theta(g(n))$
- For all values of n at and to the right of n_0, the value of $f(n)$ lies above $c_1g(n)$ and below $c_2g(n)$
- In other words: for all $n \geq n_0$ function $f(n)$ is equal to $g(n)$ to within a constant factor

$$c_1g(n) \leq f(n) \leq c_2g(n) \iff c_1 \leq \frac{f(n)}{g(n)} \leq c_2 \qquad \forall n \geq n_0$$

- $g(n)$ is an ***asymptotically tight bound*** for $f(n)$

Asymptotic non-negativity of $\Theta(g(n))$

- The definition of $\Theta(g(n))$ requires that
  - that $c_1 > 0$, $c_2 > 0$
  - $0 \leq c_1g(n) \leq c_2g(n)$ for all $n \geq n_0$
- These conditions can only be fulfilled if $g(n) \geq 0$ for all $n \geq n_0$
- Therefore, any $f(n) \in \Theta(g(n))$ must fulfill $f(n) \geq 0$ for all $n \geq n_0$
- Thus, every member $f(n) \in \Theta(g(n))$ is ***asymptotically nonnegative***
- We can thus assume that every function used within Θ-notion is asymptotically nonnegative
- This assumption holds also for the **other asymptotic notations** defined in this chapter

How to compute constants and establish order of growth?

- Introduced informal notion of Θ-notation above amounted to throwing away:
  - **lower order** terms
  - the **leading coefficient** of **higher order** terms
- How can we justify this intuition? By using the formal definition above!

Example

- Show that $\frac{1}{2}n^2 - 3n = \Theta(n^2)$
- To do so, we must determine positive constants c_1, c_2 and n_0

Recall that

$$\Theta(g(n)) = \{f(n) | \exists\, c_1 > 0, c_2 > 0, n_0 \in \mathbb{N} \ni 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)\ \forall\, n \geq n_0\}$$

We have $g(n) = n^2$ and inserting our function $\frac{1}{2}n^2 - 3n$ we get

$$c_1 n^2 \leq \frac{1}{2}n^2 - 3n \leq c_2 n^2 \tag{7}$$

for all $n \geq n_0$. Dividing by n^2 yields

$$c_1 \leq \frac{1}{2} - \frac{3}{n} \leq c_2 \tag{8}$$

Now consider **right hand** inequality only. Can we determine a positive constant c_2 that fulfills

$$\frac{1}{2} - \frac{3}{n} \leq c_2 \tag{9}$$

for all n above a certain value?

This holds for any value $n \geq 1$ by choosing any constant $c_2 \geq \frac{1}{2}$. To show this explicitly, note that

$$\frac{1}{2} - \frac{3}{n} < \frac{1}{2} \iff -\frac{3}{n} < 0 \tag{10}$$

which is true for all $n > 0$.

Now consider **left hand** inequality only. Can we determine a positive constant c_1?

$$0 < c_1 \leq \frac{1}{2} - \frac{3}{n} \tag{11}$$

The first step is to check if we can find any n so that

$$0 < \frac{1}{2} - \frac{3}{n} \tag{12}$$

By basic algebra we find

$$0 < \frac{1}{2} - \frac{3}{n} \iff n > 6 \ . \tag{13}$$

Choosing $n = 7$ we obtain

$$0 < c_1 \leq \frac{1}{2} - \frac{3}{7} = \frac{1}{14} \tag{14}$$

so the inequality holds for any value $n \geq 7$ by choosing any constant $0 \leq c_1 \leq \frac{1}{14}$.

- This shows that by choosing $c_1 = 1/14$, $c_2 = 1/2$ and $n_0 = 7$ we can verify that $\frac{1}{2}n^2 - 3n = \Theta(n^2)$
- Other choices for the constants exist, but the important thing is that *some* choices exist
- A different function belonging to $\Theta(n^2)$ would require different constants

What have we learned from this?

- For determining asymptotically tight bounds lower-term orders can be ignored for large n
- When n is large, even a tiny fraction of a higher order term suffices to dominate lower order terms
- What permits the inequalities in the Definition of Θ-notation to be satisfied?
  1. Setting c_1 to a value slightly smaller than coefficient of highest-order term
  2. Setting c_2 to a value slightly larger than coefficient of highest-order term
- The coefficient of the highest order term can likewise be ignored, since it only changes c_1 and c_2 by a constant factor equal to the coefficient

General case

- For any polynomial

  $$p(n) = \sum_{i=0}^{d} a_i n^i$$

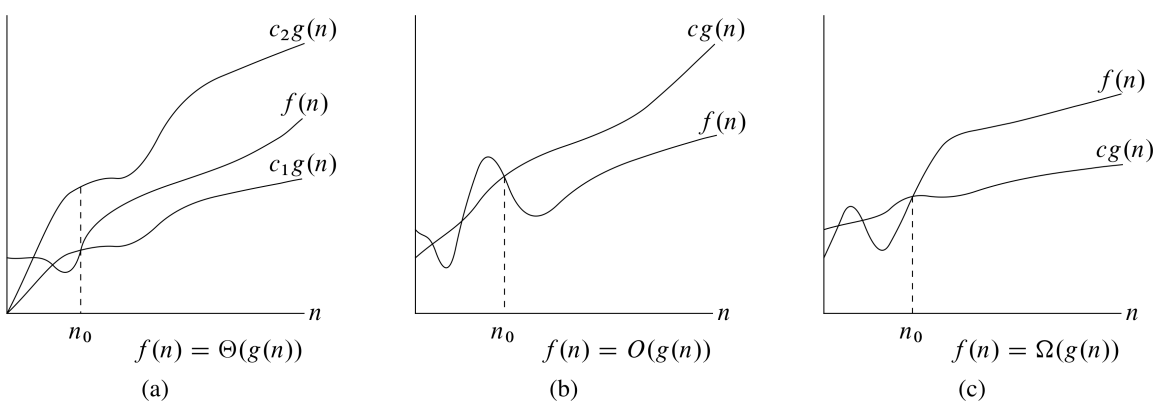
  we have $p(n) = \Theta(n^d)$, provided that
  - a_i are constants
  - $a_d > 0$
- Since any constant is a degree-0 polynominal, we can express any constant function as $\Theta(n^0)$, or $\Theta(1)$
- We shall often use notation $\Theta(1)$ to mean either a constant or a constant function with respect to some variable

O-notation

- The Θ-notation asymptotically bounds a function **from above and below**
- We use O-notation when we have only an asymptotic **upper bound**
- For a given function $g(n)$, we denote by $O(g(n))$ the set of functions

$$O(g(n)) = \{f(n) | \exists\, c > 0, n_0 \in \mathbb{N} \ni 0 \leq f(n) \leq cg(n)\ \forall\, n \geq n_0\}$$

- We use O-notation to give an upper bound on a function, to within a constant factor
- **Figure 3.1b** shows the intuition behind O-notation
- For all values n at and to the right of n_0, the function $f(n)$ is on or below $cg(n)$
- We write $f(n) = O(g(n))$ to indicate that a function $f(n)$ is a member of the set $O(g(n))$
- **Note**: $f(n) = \Theta(g(n))$ implies $f(n) = O(g(n))$, since Θ-notation is a stronger notation than O-notation
- Set theory: $\Theta(g(n)) \subseteq O(g(n))$
- Any quadratic function $an^2 + bn + c$, where $a > 0$, is in $\Theta(n^2)$ also shows that any such function is in $O(n^2)$
- **Be aware**: in literature we sometimes find O-notation informally describing asymptotically tight bounds, what we actually defined as Θ-notation
- Distinguishing asymptotically upper bound from asymptotically tight bounds is standard in the algorithms literature



Source: *CLRS Fig 3.1*

Ω-notation

- Ω-notation provides an **asymptotic lower bound**
- For a given function $g(n)$, we denote by $\Omega(g(n))$ the set of functions

$$\Omega(g(n)) = \{f(n) | \exists\, c > 0, n_0 \in \mathbb{N} \ni 0 \leq cg(n) \leq f(n)\ \forall\, n \geq n_0\}$$

- **Figure 3.1c** shows the intuition behind Ω-notation
- For all values n at or to the right of n_0, the value of $f(n)$ is on or above $cg(n)$

Theorem For any two functions $f(n)$ and $g(n)$, we have $f(n) = \Theta(g(n))$ if and only if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.

- Ω provides a lower bound on the best-case running time of an algorithm
- Example: best-case running time of insertion sort is $\Omega(n)$, which implies that the running time for insertion sort is $\Omega(n)$
- Running time of insertion sort belongs to both $\Omega(n)$ and $O(n^2)$, since it falls anywhere between a linear function of n and a quadratic function of n
- The running time of insertion sort is **not** $\Omega(n^2)$, since there exists an input for which insertion sort runs in $\Theta(n)$ time
- However: **worst-case** running time for insertion sort is $\Omega(n^2)$, since there exists an input that causes the algorithm to take $\Omega(n^2)$

Summary and extension

Asymptotic upper bound: O-notation

$$O(g(n)) = \{ f(n) | \exists c > 0, n_0 > 0 \ni 0 \leq f(n) \leq cg(n) \ \forall n \geq n_0 \} \tag{15}$$

- All $f(n)$ in $O(g(n))$ are asymptotically non-negative
- All $f(n)$ in $O(g(n))$ are asymptotically bounded *from above* by $cg(n)$
- "A function is in $O(g(n))$ if it does not grow faster than $g(n)$ for large enough n"

Examples:

- $n \in O(n)$
- $n \in O(n^2)$
- $5n^2 + 2n - 10 \in O(n^2)$
- $10^{-6}n^3 \notin O(n^2)$

Asymptotic lower bound: Ω-notation

$$\Omega(g(n)) = \{ f(n) | \exists c > 0, n_0 > 0 \ni 0 \leq cg(n) \leq f(n) \ \forall n \geq n_0 \} \tag{16}$$

- All $f(n)$ in $\Omega(g(n))$ are asymptotically non-negative
- All $f(n)$ in $\Omega(g(n))$ are asymptotically bounded *from below* by $cg(n)$
- "A function is in $\Omega(g(n))$ if it grows at least as fast as $g(n)$ for large enough n"

Examples:

- $n \in \Omega(n)$
- $n^2 \in \Omega(n)$
- $5n^2 + 2n - 10 \in \Omega(n^2)$
- $10^6 n^2 \notin \Omega(n^3)$

Asymptotic tight bound: Θ-notation

$$\Theta(g(n)) = \{ f(n) | \exists c_1 > 0, c_2 > 0, n_0 > 0 \ni 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \ \forall n \geq$$

- All $f(n)$ in $\Theta(g(n))$ are asymptotically non-negative
- All $f(n)$ in $\Theta(g(n))$ are asymptotically bounded *from above and below* by $c_1 g(n)$ and $c_2 g(n)$
- "A function is in $\Theta(g(n))$ if it grows as fast as $g(n)$ for large enough n

Examples:

- $n \in \Theta(n)$
- $5n^2 + 2n - 10 \in \Theta(n^2)$
- $5n^2 + 2n - 10 \notin \Theta(n)$ (grows too fast, upper bound violated)
- $5n^2 + 2n - 10 \notin \Theta(n^3)$ (grows too slowly, lower bound violated)

Relations between O, Ω, Θ

- $\Theta(g(n)) \subseteq O(g(n))$
- $\Theta(g(n)) \subseteq \Omega(g(n))$

Theorem 3.1

Let $f(n)$ and $g(n)$ be any two functions. Then

$$\begin{aligned} f(n) \in \Theta(g(n)) \iff & f(n) \in O(g(n)) \wedge f(n) \in \Omega(g(n)) && (18) \\ \iff & \text{ if and only if} && (19) \\ \wedge & \text{ and} && (20) \end{aligned}$$

Simplified notation

Instead of

$$f(n) \in \Theta(g(n))$$

we usually write

$$f(n) = \Theta(g(n))$$

even though this is a slight abuse of the equal sign.

Asymptotically tight bound: Worked example

$$f(n) = \frac{n^2}{2} - 2n$$

Find c_1, c_2, and n_0 so that

$$c_1 n^2 \leq f(n) \leq c_2 n^2 \qquad \forall n \geq n_0 \tag{21}$$

$$\iff c_1 n^2 \leq \frac{n^2}{2} - 2n \leq c_2 n^2 \tag{22}$$

$$\iff c_1 \leq \frac{1}{2} - \frac{2}{n} \leq c_2 \tag{23}$$

- Since $c_1 > 0$ and $c_2 > 0$ required, we must have $n > 4$, i.e., $n_0 > 4$.
- We choose $n_0 = 8$ for convenience

We then have

$$c_1 \leq \frac{1}{2} - \frac{2}{8} = \frac{1}{4}$$

and

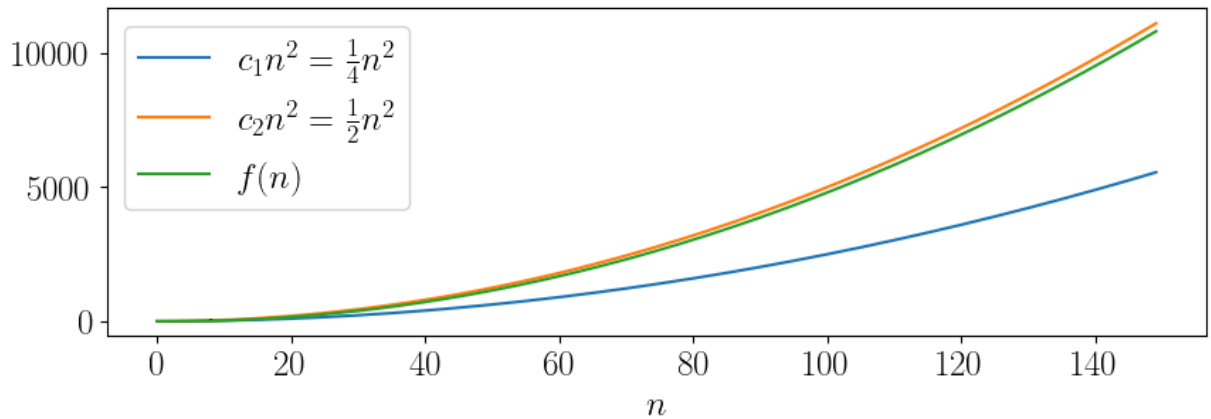
$$c_2 \geq \lim_{n \to \infty} \frac{1}{2} - \frac{2}{n} = \frac{1}{2} \ .$$

We thus choose $n_0 = 8$, $c_1 = 1/4$, $c_2 = 1/2$ and conclude that $f(n) = \Theta(n^2)$.

In [8]:
```
import matplotlib.pyplot as plt
import numpy as np
```

In [9]:
```
plt.rcParams['figure.figsize'] = (10, 3)
plt.rcParams['text.usetex'] = True
plt.rcParams['font.size'] = 18
```

In [10]:
```
n = np.arange(150)
plt.plot(n, 0.25*n**2, label=r'$c_1 n^2=\frac{1}{4} n^2$')
plt.plot(n, 0.5*n**2, label=r'$c_2 n^2 = \frac{1}{2} n^2$')
plt.plot([8, 8], [0, 100], 'k--')
plt.plot(n, 0.5*n**2-2*n, label=r'$f(n)$')
plt.xlabel(r'$n$')
plt.legend();
```



O, Ω and Θ in equations

- In equations, $\Theta(g(n))$ stands for any function from this set

$$2n^2 + 3n + 1 = 2n^2 + \Theta(n)$$

means

$$2n^2 + 3n + 1 = 2n^2 + f(n)$$

for some $f(n) \in \Theta(n)$; we just don't care about the details and don't even want to name the function.

- If asymptotic expressions appear on both sides, e.g.,

$$2n^2 + \Theta(n) = \Theta(n^2)$$

we interprete this as saying: for any $f(n) \in \Theta(n)$ on the left-hand side, there is a $g(n) \in \Theta(n^2)$, so that $2n^2 + f(n) = g(n)$.

- In particular, we can simplify

$$2n^2 + 3n + 1 = 2n^2 + \Theta(n) = \Theta(n^2)\ .$$

Properties of asymptotic functions

Transitivity (Θ, O, Ω)

$$f(n) = \Theta(g(n)) \wedge g(n) = \Theta(h(n)) \implies f(n) = \Theta(h(n))$$

Reflexivity (Θ, O, Ω)

$$f(n) = \Theta(f(n))$$

Symmetry (Θ)

$$f(n) = \Theta(g(n)) \iff g(n) = \Theta(f(n))$$

Transpose symmetry (O, Ω)

$$f(n) = O(g(n)) \iff g(n) = \Omega(f(n)) \tag{24}$$

Comparison

We have roughly

$\leq$	$=$	$\geq$
O	Θ	Ω

Note: For numbers a and b, one of the following must be true: $a < b$, $a = b$ or $a > b$. For functions, this may be undecided.

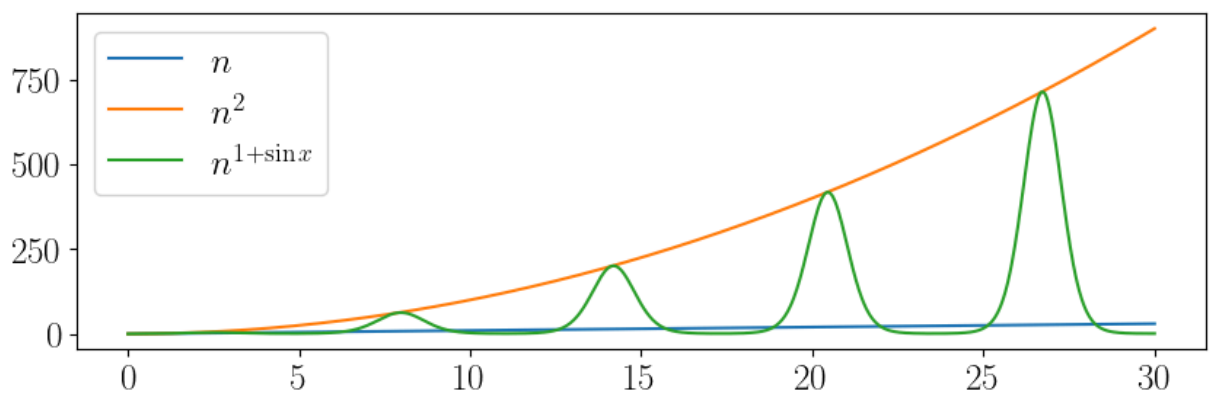
Example

The function

$$f(n) = n^{1+\sin n}$$

is neither asymptotically larger nor asymptotically smaller than $g(n) = n$ because the exponent in $f(n)$ varies between 0 and 2.

In [11]:
```
x = np.linspace(0, 30, num=1000)
plt.plot(x, x, label='$n$');
plt.plot(x, x**2, label='$n^2$');
plt.plot(x, x**(1+np.sin(x)), label=r'$n^{1+\sin x}$');
plt.legend();
```



Solving recurrence equations

See Compendium, part 5. We will only consider one approach to solving recurrence equations here.

Examples of recurrences with solutions

Example 1

$$T(n) = \begin{cases} 1 & \text{if} \quad n = 1 \\ T(n-1) + 1 & \text{if} \quad n > 1 \end{cases}$$

Solution: $T(n) = n$

Example 2

$$T(n) = \begin{cases} 1 & \text{if} \quad n = 1 \\ 2T(n/2) + n & \text{if} \quad n > 1 \end{cases}$$

Solution: $T(n) = n \lg n + n$

Proof by induction

Base case

$$n = 1 \implies n \lg n + n = 1 \lg 1 + 1 = 1 = T(1) \ .$$

Inductive step

1. We assume (*inductive hypothesis*) that $T(k) = k \lg k + k \ \forall k < n$.
2. We then have

$$\begin{aligned} T(n) &= 2T\left(\frac{n}{2}\right) + n \qquad \text{by recurrence} && (25) \\ &= 2\left(\frac{n}{2} \lg \frac{n}{2} + \frac{n}{2}\right) + n \qquad \text{by inductive hypothesis} && (26) \\ &= n \lg \frac{n}{2} + 2n && (27) \\ &= n(\lg n - \lg 2) + 2n \qquad \text{by properties of the logarithm} && (28) \\ &= n \lg n - n \lg 2 + 2n && (29) \\ &= n \lg n - n + 2n \qquad \text{because } \lg 2 = 1 \text{ by definition} && (30) \\ &= n \lg n + n \ . && (31) \end{aligned}$$

3. We have thus proven by induction that the solution holds for all n.

Solving recurrences using the "Master method"

General solution to recurrences of the form

$$T(n) = aT(n/b) + f(n) \qquad \text{where } a \geq 1, b > 1, f(n) > 0.$$

- Useful to solve many recurrences, but not all
- Note that a and b must be constants, *independent* of n

- $f(n)$ needs to fulfill certain requirements
- Solution has three cases, depending on how $f(n)$ relates to $n^{\log_b a}$

Case 1

Condition

$$f(n) = O(n^{\log_b a - \epsilon}) \quad \text{for some } \epsilon > 0$$

- $f(n)$ is polynomially smaller than $n^{\log_b a}$

Solution

$$T(n) = \Theta(n^{\log_b a})$$

- Runtime is dominated by leaves of recursion tree, internal operations contribute little

Case 2

Condition

$$f(n) = \Theta(n^{\log_b a})$$

- $f(n)$ grows as fast as $n^{\log_b a}$

Solution

$$T(n) = \Theta(n^{\log_b a} \lg n)$$

- Recursion tree has $\Theta(\lg n)$ levels, equal cost $\sim n$ at each level
- This case can be extended to $f(n) = \Theta(n^{\log_b a} \lg^k n)$, but that is not part of the curriculum for INF221 (for details, see, e.g., Wikipedia))

Case 3

Condition

$$f(n) = \Omega(n^{\log_b a + \epsilon}) \quad \text{for some } \epsilon > 0$$

and regularity condition

$$af(n/b) \leq cf(n) \quad \text{for some } c < 1 \text{ and all large } n$$

- $f(n)$ is polynomially greater than $n^{\log_b a}$
- Regularity condition always fulfilled if $f(n) = n^k$ is polynomial

Solution

$$T(n) = \Theta(f(n))$$

- Runtime dominated by root of recursion tree

Examples where master theorem can be applied

$$T(n) = aT(n/b) + f(n) \qquad \text{where } a \geq 1, b > 1, f(n) > 0.$$

Example 1

- $T(n) = 5T(n/2) + \Theta(n^2) \implies a = 5, b = 2$
- Compare $n^{\log_2 5}$ to n^2
- We have $\log_2 5 > 2 \implies \log_2 5 - \epsilon = 2 \implies$ Case 1
- $T(n) = \Theta(n^{\lg 5})$

Example 2

- $T(n) = 27T(n/3) + \Theta(n^3) \implies a = 27, b = 3$
- Compare $n^{\log_3 27} = n^3$ to n^3
- Case 2
- $T(n) = \Theta(n^3 \lg n)$

Example 3

- $T(n) = 5T(n/2) + \Theta(n^3) \implies a = 5, b = 2$
- Compare $n^{\log_2 5}$ to n^3
- We have $\log_2 5 < 3 \implies \log_2 5 + \epsilon = 3 \implies$ Case 3
- Regularity condition: $af(n/b) = 5(n/2)^3 = 5n^3/8 \leq cn^3$ for $c = 5/8 < 1$
- Case 3
- $T(n) = \Theta(n^3)$

Examples where master theorem cannot be applied

Example 1: non-constant a

- $T(n) = 2^n T(n/4) + \Theta(1) \implies a = 2^n, b = 4$
- a is not constant, so theorem does not apply

Example 2: regularity condition not fulfilled

- $T(n) = 2T(n/2) + n^2(2 - \cos n) \implies a = 2, b = 2, f(n) = n^2(2 - \cos n)$
- We have $n^{\log_b a} = n$ and $f(n) = n^2(2 - \cos n) \geq n^2$ because $-1 \leq \cos n \leq 1$
- Therefore, only case 3 may apply: we have $f(n) = \Omega(n^2)$, which fits the requirement for case 3 with $\epsilon = 1$
- Now consider the regularity condition:
  - We need to show that $af(n/b) \leq cf(n)$ for some fixed $c < 1$ and **all** $n > n_0$
  - We will show that we can for any n_0, always find some n that will break this condition

    - Choose some large n so that $n \approx 2(2k+1)\pi$
    - Thus $\cos n \approx 1 \implies f(n) \approx n^2$
    - We further have $af(n/b) = 2f((2k+1)\pi)$ and $\cos((2k+1)\pi) \approx -1$
    - Therefore $af(n/b) \approx 2(n/2)^2(2-(-1)) = 6n^2/4 = \frac{3}{2}n^2$
    - The regularity condition then becomes $\frac{3}{2}n^2 \leq cn^2$
    - So for this n, we would need $c \geq \frac{3}{2}$, violating the requirement that $c < 1$. Since we clearly can find some $n \approx 2(2k+1)\pi$ everywhere along the number line, there is no n_0 beyond which the regularity condition would hold, and case 3 does not apply.
- Note that this is a pretty artifically constructed case.

Some useful mathematical material

Positivity

- Positive (non-negative): ≥ 0
- Strictly positive: > 0

Monotonicity

- $f(n)$ is *monotonically increasing* if $m \leq n \implies f(m) \leq f(n)\ \forall m, n$
- $f(n)$ is *strictly (monotonically) increasing* if $m < n \implies f(m) < f(n)\ \forall m, n$
- correspondingly for *decreasing*

Floor and Ceiling

- Floor: $\lfloor x \rfloor$ is greatest integer $\leq x$
- Ceiling: $\lceil x \rceil$ is smallest integer $\geq x$

$$x - 1 < \lfloor x \rfloor \leq x \leq \lceil x \rceil < x + 1$$

For any integer n

$$\lfloor n/2 \rfloor + \lceil n/2 \rceil = n$$

Modulo

For $a \in \mathbb{Z}$ and $n \in \mathbb{N}$, we define

$$a \bmod n = a - n \left\lfloor \frac{a}{n} \right\rfloor$$

i.e., $a \bmod n$ is the remainder upon dividing a by n.

Exponentials

$$a^{-1} = 1/a \tag{32}$$
$$(a^m)^n = a^{mn} \tag{33}$$
$$a^m a^n = a^{m+n} \tag{34}$$
$$e^x \geq 1 + x \tag{35}$$
$$\lim_{n\to\infty} \left(1 + \frac{x}{n}\right)^n = e^x \tag{36}$$
$$0^0 = 1 \qquad \text{by definition} \tag{37}$$

Logarithms

$$\lg n = \log_2 n \qquad \text{(binary log)} \tag{38}$$
$$\ln n = \log_e n \qquad \text{(natural log)} \tag{39}$$
$$\lg^k n = (\lg n)^k \tag{40}$$
$$\lg \lg n = \lg(\lg n) \tag{41}$$
$$a = b^{\log_b a} \tag{42}$$
$$\log_c(ab) = \log_c a + \log_c b \tag{43}$$
$$\log_b a^n = n \log_b a \tag{44}$$
$$\log_b a = \frac{\log_c a}{\log_c b} = \frac{\ln a}{\ln b} = \frac{\lg a}{\lg b} \tag{45}$$
$$\log_b a = \frac{1}{\log_a b} \tag{46}$$
$$a^{\log_b c} = c^{\log_b a} \tag{47}$$

Factorials

$$n! = 1 \times 2 \times 3 \times \cdots \times n \tag{48}$$
$$0! = 1 \tag{49}$$

Stirling's approximation of the factorial

$$n! = \sqrt{2\pi n}\left(\frac{n}{e}\right)^n \left(1 + \Theta\left(\frac{1}{n}\right)\right) \tag{50}$$

Logarithm of the factorial

$$\lg n! = \Theta(n \lg n) \tag{51}$$

Sums of integers

$$\sum_{k=1}^{n} k = \frac{n(n+1)}{2} \tag{52}$$

$$\sum_{k=1}^{n} k^2 = \frac{n(n+1)(2n+1)}{6} \tag{53}$$

$$\sum_{k=1}^{n} k^3 = \frac{n^2(n+1)^2}{4} \tag{54}$$

Geometric series

For $x \neq 1$

$$\sum_{k=0}^{n} x^k = 1 + x + x^2 + \cdots + x^n = \frac{x^{n+1} - 1}{x - 1} \tag{55}$$

is a geometric series. For $|x| < 1$ and $n \to \infty$ we have

$$\sum_{k=0}^{\infty} x^k = \lim_{n\to\infty} \frac{x^{n+1} - 1}{x - 1} = \frac{1}{1 - x} \ . \tag{56}$$

In [ ]: